# EN3160 Assignment 3: Neural Networks

**Name:** KPCP Keerawella
**Index number:** 230332B
**GitHub profile:** https://github.com/chalee88

This notebook contains the three parts of the assignment: a dense network for CIFAR-10, two CNNs for MNIST, and transfer learning to classify ants and bees. The results below use the full training and evaluation sets.

```
import os, random, time, copy, json
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms, models
from torchvision.datasets.utils import download_and_extract_archive
from IPython.display import display, Markdown

ROOT = Path.cwd()
DATA = ROOT / 'data'
OUT = ROOT / 'results'
OUT.mkdir(exist_ok=True)
os.environ['TORCH_HOME'] = str(DATA / 'torch')
SEED = 42
def seed_all():
    random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
seed_all()
torch.set_num_threads(4)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('PyTorch:', torch.__version__, '| device:', device)
if device.type == 'cuda': print(torch.cuda.get_device_name(0))
def loader(ds, batch=128, shuffle=False):
    return DataLoader(ds, batch_size=batch, shuffle=shuffle,
num_workers=0,
                      pin_memory=device.type == 'cuda' and not
(isinstance(ds, TensorDataset) and ds.tensors[0].is_cuda),
                      generator=torch.Generator().manual_seed(SEED))

def cache_fixed(ds):
    # Store preprocessed images on the GPU to avoid repeating this
```

```
work.
    if device.type != 'cuda': return ds
    batches = list(DataLoader(ds, batch_size=512, shuffle=False,
num_workers=0))
    x = torch.cat([b[0] for b in batches]).to(device)
    y = torch.cat([b[1] for b in batches]).to(device)
    return TensorDataset(x, y)

def curves(history, title, eval_label='test'):
    h = pd.DataFrame(history)
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.3))
    for ax, metric in zip(axes, ['loss', 'accuracy']):
        for split in ['train', 'test']:
            ax.plot(h.epoch, h[f'{split}_{metric}'], marker='o',
label=split if split == 'train' else eval_label)
        ax.set(xlabel='Epoch', ylabel=metric.title(), title=title + ':
' + metric)
        ax.legend(); ax.grid(alpha=.25)
    fig.tight_layout(); plt.show()

def confusion(matrix, labels, title):
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(matrix, cmap='Blues')
    for i in range(len(labels)):
        for j in range(len(labels)):
            ax.text(j, i, str(matrix[i,j]), ha='center', va='center',
                    color='white' if matrix[i,j] > matrix.max()/2 else
'black', fontsize=8)
    ax.set(xticks=range(len(labels)), yticks=range(len(labels)),
           xticklabels=labels, yticklabels=labels, xlabel='Predicted
label',
           ylabel='True label', title=title)
    fig.colorbar(im, ax=ax); fig.tight_layout(); plt.show()

PyTorch: 2.5.1+cu121 | device: cuda
NVIDIA GeForce RTX 4060 Laptop GPU
```

# 1. Manual forward and backward propagation on CIFAR-10

Each CIFAR-10 image has $32 \times 32 \times 3 = 3072$ input values. I added a hidden layer with 100 sigmoid neurons before the existing 10-class output layer. The network is therefore $3072 \to 100 \to 10$.

For a batch of $N$ images, the forward pass is

$$H = \sigma\left(X W_1 + b_1\right), Z = H W_2 + b_2.$$

Here, $H$ contains the 100 hidden activations for each image, and $Z$ contains its 10 class scores. Softmax converts these scores into probabilities. The average cross-entropy loss is

$$L = -\frac{1}{N}\sum_i \log softmax(Z_i)_{y_i}.$$

I calculated the gradients manually using the chain rule. With one-hot labels $Y$, the output-layer gradients are

$$G = (softmax(Z) - Y)/N, \nabla W_2 = H^T G, \nabla b_2 = \sum_i G_i.$$

The hidden-layer gradients include the sigmoid derivative $H(1-H)$:

$$D = (G W_2^T) \odot H \odot (1-H), \nabla W_1 = X^T D, \nabla b_1 = \sum_i D_i.$$

All gradients are calculated before updating any weights. The code uses `logsumexp` to calculate cross-entropy without numerical overflow. PyTorch autograd is used only to check the manual gradients, not to train this network.

I trained for 10 epochs with batch size 50 and no regularization. I used an initial learning rate of 0.1 and multiplied it by 0.9 after each epoch. I also used Xavier-style initialization. These are changes from the starter code and were chosen as the settings for this run; I did not compare different learning rates or initializations.

```
cifar_transform = transforms.Compose([transforms.ToTensor(),
transforms.Normalize((.5,)*3, (.5,)*3)])
cifar_train = datasets.CIFAR10(DATA, train=True, download=True,
transform=cifar_transform)
cifar_test = datasets.CIFAR10(DATA, train=False, download=True,
transform=cifar_transform)
cifar_train, cifar_test = cache_fixed(cifar_train),
cache_fixed(cifar_test)
cifar_loaders = {'train': loader(cifar_train, 50, True), 'test':
loader(cifar_test, 256)}
seed_all()
W1 = torch.randn(3072, 100, device=device) * np.sqrt(2/(3072+100))
b1 = torch.zeros(100, device=device)
W2 = torch.randn(100, 10, device=device) * np.sqrt(2/110)
b2 = torch.zeros(10, device=device)

def manual_forward(x, params):
    w1, bias1, w2, bias2 = params
    h = torch.sigmoid(x @ w1 + bias1)
    return h, h @ w2 + bias2

def manual_gradients(x, y, params):
    h, z = manual_forward(x, params)
    loss = (torch.logsumexp(z, dim=1) - z[torch.arange(len(y),
device=y.device), y]).mean()
    g = z.softmax(dim=1)
    g[torch.arange(len(y), device=y.device), y] -= 1
    g /= len(y)
```

```
    d = (g @ params[2].T) * h * (1-h)
    return loss, [x.T @ d, d.sum(0), h.T @ g, g.sum(0)]

# Check the manual gradients against autograd. Training still uses the
manual gradients.
check_params = [p.detach().double().clone().requires_grad_() for p in
[W1,b1,W2,b2]]
xcheck = torch.randn(4, 3072, device=device, dtype=torch.float64)
ycheck = torch.tensor([0,3,5,9], device=device)
_, zcheck = manual_forward(xcheck, check_params)
reference = torch.autograd.grad(nn.functional.cross_entropy(zcheck,
ycheck), check_params)
with torch.no_grad():
    _, analytical = manual_gradients(xcheck, ycheck, check_params)
    errors = [float((a-b).abs().max()) for a,b in zip(reference,
analytical)]
assert max(errors) < 1e-10
print('Gradient check maximum absolute errors:', errors)

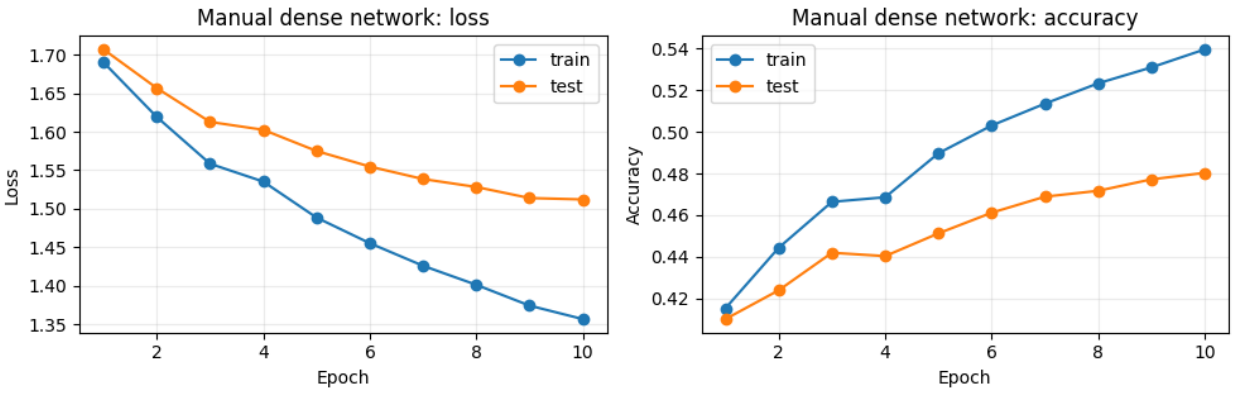
@torch.no_grad()
def manual_evaluate(dl):
    total_loss = correct = count = 0
    for x,y in dl:
        x,y = x.to(device).flatten(1), y.to(device)
        _,z = manual_forward(x, [W1,b1,W2,b2])
        total_loss +=
nn.functional.cross_entropy(z,y,reduction='sum').item()
        correct += (z.argmax(1)==y).sum().item(); count += len(y)
    return total_loss/count, correct/count

manual_history = []
with torch.no_grad():
    for epoch in range(1,11):
        lr = .1 * .9**(epoch-1)
        for x,y in cifar_loaders['train']:
            x,y = x.to(device).flatten(1),y.to(device)
            _, grads = manual_gradients(x,y,[W1,b1,W2,b2])
            for p,g in zip([W1,b1,W2,b2],grads): p.add_(g, alpha=-lr)
        tr = manual_evaluate(cifar_loaders['train'])
        te = manual_evaluate(cifar_loaders['test'])
        row =
dict(epoch=epoch,train_loss=tr[0],train_accuracy=tr[1],test_loss=te[0]
,test_accuracy=te[1])
        manual_history.append(row); print(row, flush=True)
pd.DataFrame(manual_history).to_csv(OUT/'manual_cifar10.csv',
index=False)
curves(manual_history, 'Manual dense network')
display(pd.DataFrame(manual_history).tail(1))
```

```
Downloading https://cave.cs.toronto.edu/kriz/cifar-10-python.tar.gz to
c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\cifar-10-
python.tar.gz

100%|██████████| 170M/170M [1:09:10<00:00, 41.1kB/s]

Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\
data\cifar-10-python.tar.gz to c:\Users\Chaleesha\Documents\2026\Sem
5\CV\assignment 3\data
Files already downloaded and verified
Gradient check maximum absolute errors: [2.7755575615628914e-17,
6.938893903907228e-18, 4.163336342344337e-17, 8.326672684688674e-17]
{'epoch': 1, 'train_loss': 1.6900974397277833, 'train_accuracy':
0.41536, 'test_loss': 1.7069540559768677, 'test_accuracy': 0.4102}
{'epoch': 2, 'train_loss': 1.619198286895752, 'train_accuracy':
0.4444, 'test_loss': 1.6566701969146729, 'test_accuracy': 0.424}
{'epoch': 3, 'train_loss': 1.5584855765533447, 'train_accuracy':
0.46642, 'test_loss': 1.6127866903305055, 'test_accuracy': 0.442}
{'epoch': 4, 'train_loss': 1.5352719129180907, 'train_accuracy':
0.46862, 'test_loss': 1.6025387685775756, 'test_accuracy': 0.4404}
{'epoch': 5, 'train_loss': 1.4885477402496339, 'train_accuracy':
0.4897, 'test_loss': 1.5750037281036378, 'test_accuracy': 0.4513}
{'epoch': 6, 'train_loss': 1.455280304260254, 'train_accuracy':
0.50302, 'test_loss': 1.5547842903137208, 'test_accuracy': 0.4612}
{'epoch': 7, 'train_loss': 1.425955573501587, 'train_accuracy':
0.5135, 'test_loss': 1.5385887714385986, 'test_accuracy': 0.4689}
{'epoch': 8, 'train_loss': 1.4012337689971923, 'train_accuracy':
0.52324, 'test_loss': 1.5282821487426759, 'test_accuracy': 0.4717}
{'epoch': 9, 'train_loss': 1.3742305813598632, 'train_accuracy':
0.53088, 'test_loss': 1.513907905960083, 'test_accuracy': 0.4772}
{'epoch': 10, 'train_loss': 1.3567232077789306, 'train_accuracy':
0.5395, 'test_loss': 1.5121122234344482, 'test_accuracy': 0.4803}
```




| | epoch | train_loss | train_accuracy | test_loss | test_accuracy |
|---|---|---|---|---|---|
| 9 | 10 | 1.356723 | 0.5395 | 1.512112 | 0.4803 |

## 2. LeNet-5 and a modern CNN on MNIST

I trained both models on the same 60,000 training images and evaluated them on the same 10,000 test images. Both use 32 x 32 inputs, the same normalization, batch size 128, Adam with learning rate 0.001, no weight decay, and 10 epochs. I used the same seed and did not apply data augmentation.

The LeNet-5 model uses two 5 x 5 convolution layers with 6 and 16 feature maps, average pooling, tanh activations, and dense layers with 120 and 84 neurons. This is a common adaptation of LeNet-5: the second convolution layer uses all input channels, and the output gives 10 class scores rather than using the original sparse connections and RBF classifier.

The modern CNN has three convolution layers with 32, 64 and 128 feature maps. Each is followed by batch normalization and ReLU. It also uses dropout and global average pooling.

After each epoch, I measured loss and accuracy on the full training and test sets in evaluation mode. This turns off dropout and uses the stored batch-normalization statistics. I plotted all four metrics and displayed a test confusion matrix for each model. The test results were not used to choose a model checkpoint or change the training settings.

```
mnist_transform = transforms.Compose([transforms.Resize((32,32)),
transforms.ToTensor(),
                                      transforms.Normalize((.1307,),
(.3081,))])
mnist_train = datasets.MNIST(DATA, train=True, download=True,
transform=mnist_transform)
mnist_test = datasets.MNIST(DATA, train=False, download=True,
transform=mnist_transform)
mnist_train, mnist_test = cache_fixed(mnist_train),
cache_fixed(mnist_test)

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Conv2d(1,6,5), nn.Tanh(),
nn.AvgPool2d(2),
            nn.Conv2d(6,16,5), nn.Tanh(), nn.AvgPool2d(2),
nn.Flatten(),
            nn.Linear(16*5*5,120), nn.Tanh(), nn.Linear(120,84),
nn.Tanh(), nn.Linear(84,10))
    def forward(self,x): return self.net(x)

class ModernCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1,32,3,padding=1,bias=False),
nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1,bias=False),
nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
```

```
            nn.Dropout2d(.2),
            nn.Conv2d(64,128,3,padding=1,bias=False),
nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(.5),
nn.Linear(128,10))
    def forward(self,x): return self.net(x)

@torch.no_grad()
def evaluate(model, dl, classes):
    model.eval(); loss = correct = count = 0
    cm = torch.zeros(classes, classes, dtype=torch.int64)
    for x,y in dl:
        x,y = x.to(device),y.to(device)
        z = model(x); pred = z.argmax(1)
        loss +=
nn.functional.cross_entropy(z,y,reduction='sum').item()
        correct += (pred==y).sum().item(); count += len(y)
        cm += torch.bincount((y*classes+pred).cpu(),
minlength=classes**2).reshape(classes,classes)
    return loss/count, correct/count, cm.numpy()

def train_model(model, train_dl, train_eval_dl, test_dl, epochs,
optimizer, classes,
                frozen=False, scheduler=None, name='model'):
    history=[]; start=time.perf_counter()
    for epoch in range(1, epochs+1):
        model.train()
        # Keep batch-normalization statistics fixed when the backbone
is frozen.
        if frozen:
            model.eval(); model.fc.train()
        for x,y in train_dl:
            x,y=x.to(device),y.to(device)
            optimizer.zero_grad(set_to_none=True)
            loss=nn.functional.cross_entropy(model(x),y)
            loss.backward(); optimizer.step()
        tr=evaluate(model,train_eval_dl,classes);
te=evaluate(model,test_dl,classes)

row=dict(epoch=epoch,train_loss=tr[0],train_accuracy=tr[1],test_loss=t
e[0],test_accuracy=te[1])
        history.append(row); print(name, row, flush=True)
        pd.DataFrame(history).to_csv(OUT/(name+'.csv'),index=False)
        if scheduler is not None: scheduler.step()
    elapsed=time.perf_counter()-start
    torch.save(model.state_dict(),OUT/(name+'.pt'))
    return history, te[2], elapsed

mnist_results={}
for name, constructor in [('lenet5',LeNet5),('modern_cnn',ModernCNN)]:
```

```
    seed_all()
    model=constructor().to(device)
    print(name, 'parameters:',sum(p.numel() for p in
model.parameters()))

result=train_model(model,loader(mnist_train,128,True),loader(mnist_tra
in,256),

loader(mnist_test,256),10,torch.optim.Adam(model.parameters(),lr=.001)
,10,name=name)
    mnist_results[name]=result
    curves(result[0],name)
    confusion(result[1],list(range(10)),name+' test confusion matrix')
    assert result[1].sum()==len(mnist_test)
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-
ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-
images-idx3-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-
images-idx3-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\
assignment 3\data\MNIST\raw\train-images-idx3-ubyte.gz

100%|██████████| 9.91M/9.91M [00:11<00:00, 873kB/s]

Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\
data\MNIST\raw\train-images-idx3-ubyte.gz to c:\Users\Chaleesha\
Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw

Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-
ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-
labels-idx1-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-
labels-idx1-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\
assignment 3\data\MNIST\raw\train-labels-idx1-ubyte.gz

100%|██████████| 28.9k/28.9k [00:00<00:00, 72.2kB/s]

Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\
data\MNIST\raw\train-labels-idx1-ubyte.gz to c:\Users\Chaleesha\
Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
Failed to download (trying next):
```

```
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw\t10k-images-idx3-ubyte.gz

100%|██████████| 1.65M/1.65M [00:02<00:00, 637kB/s]

Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw\t10k-images-idx3-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

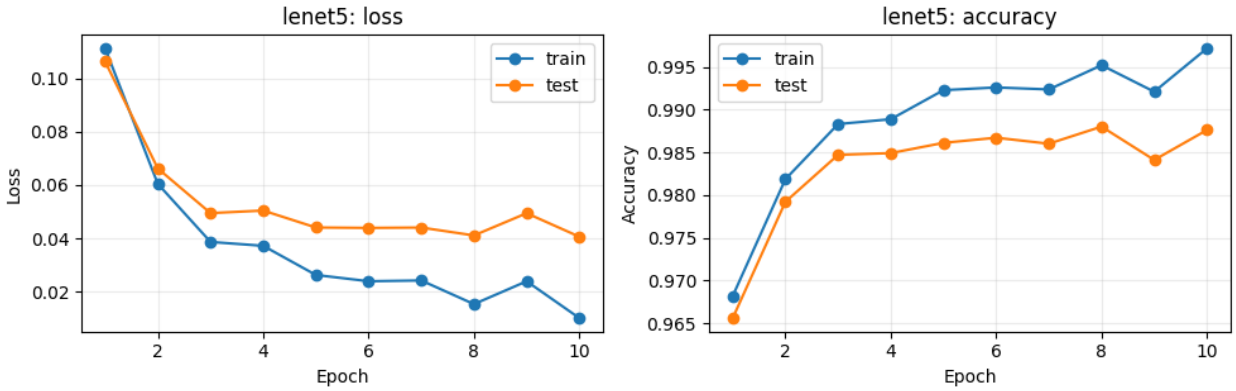
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw\t10k-labels-idx1-ubyte.gz

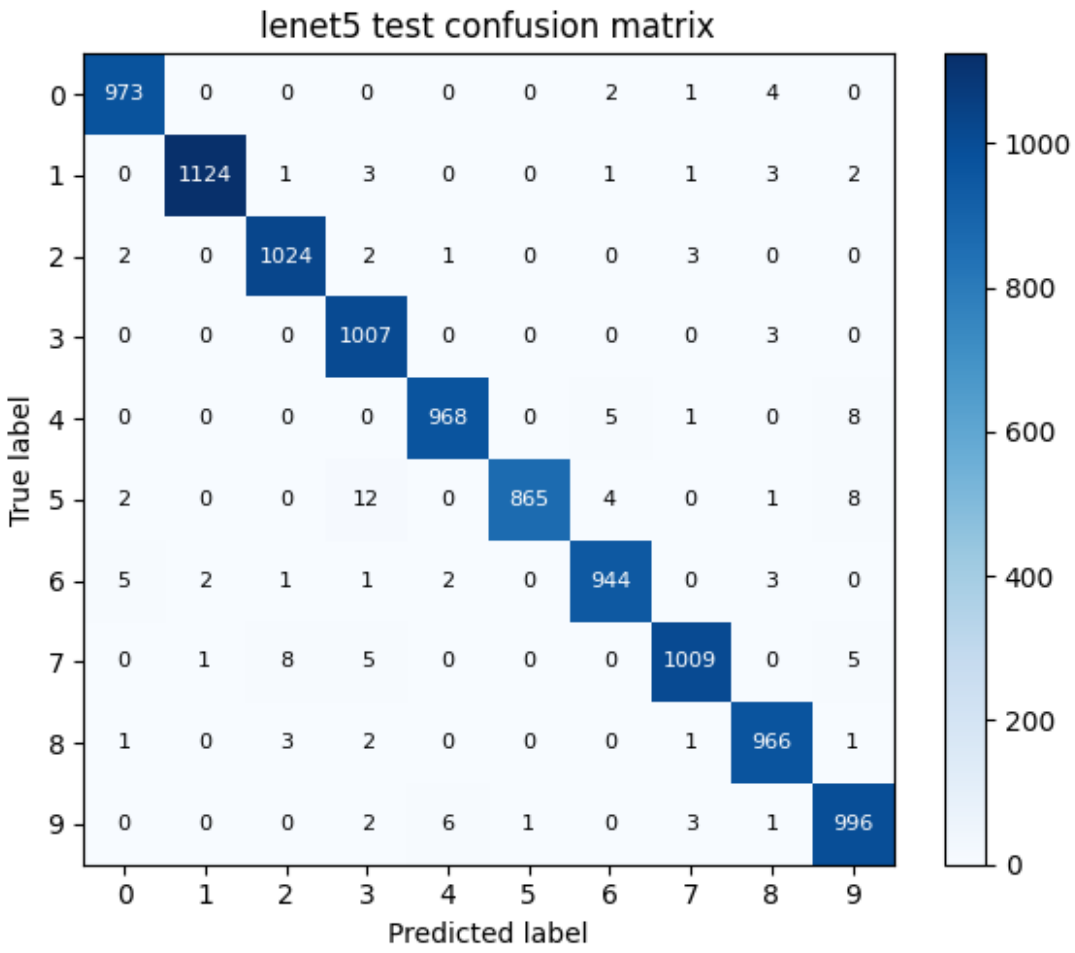
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.71MB/s]

Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw\t10k-labels-idx1-ubyte.gz to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\MNIST\raw

lenet5 parameters: 61706
lenet5 {'epoch': 1, 'train_loss': 0.11115678486824036,
'train_accuracy': 0.9681833333333333, 'test_loss':
0.10619743360579013, 'test_accuracy': 0.9656}
lenet5 {'epoch': 2, 'train_loss': 0.06038278032342593,
'train_accuracy': 0.98185, 'test_loss': 0.06615689109042287,
'test_accuracy': 0.9792}
lenet5 {'epoch': 3, 'train_loss': 0.038628211787839734,
'train_accuracy': 0.9883, 'test_loss': 0.049431482191570106,
'test_accuracy': 0.9847}
lenet5 {'epoch': 4, 'train_loss': 0.03717156898379326,
'train_accuracy': 0.9888666666666667, 'test_loss':
0.05038879799023271, 'test_accuracy': 0.9849}
lenet5 {'epoch': 5, 'train_loss': 0.026246701967716217,
'train_accuracy': 0.9922666666666666, 'test_loss':
0.04403311923732981, 'test_accuracy': 0.9861}
lenet5 {'epoch': 6, 'train_loss': 0.023896956491221984,
'train_accuracy': 0.9925833333333334, 'test_loss':
0.04388019165648147, 'test_accuracy': 0.9867}
lenet5 {'epoch': 7, 'train_loss': 0.02418949962258339,
'train_accuracy': 0.99235, 'test_loss': 0.04401727176820859,
```

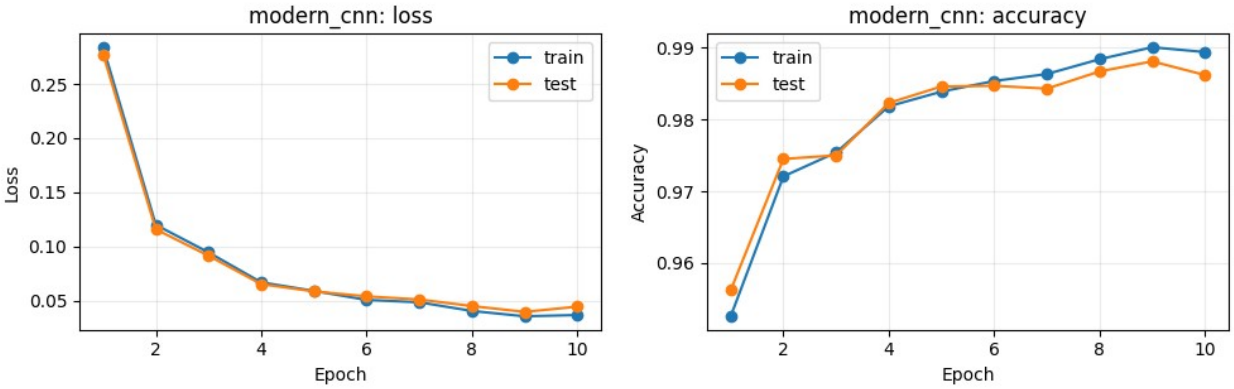
```
'test_accuracy': 0.986}
lenet5 {'epoch': 8, 'train_loss': 0.015318304465214412,
'train_accuracy': 0.9951833333333333, 'test_loss':
0.04105832621050067, 'test_accuracy': 0.988}
lenet5 {'epoch': 9, 'train_loss': 0.02388709032976379,
'train_accuracy': 0.9920833333333333, 'test_loss':
0.04939404792226851, 'test_accuracy': 0.9841}
lenet5 {'epoch': 10, 'train_loss': 0.010085548052564263,
'train_accuracy': 0.99715, 'test_loss': 0.04058121976578259,
'test_accuracy': 0.9876}
```

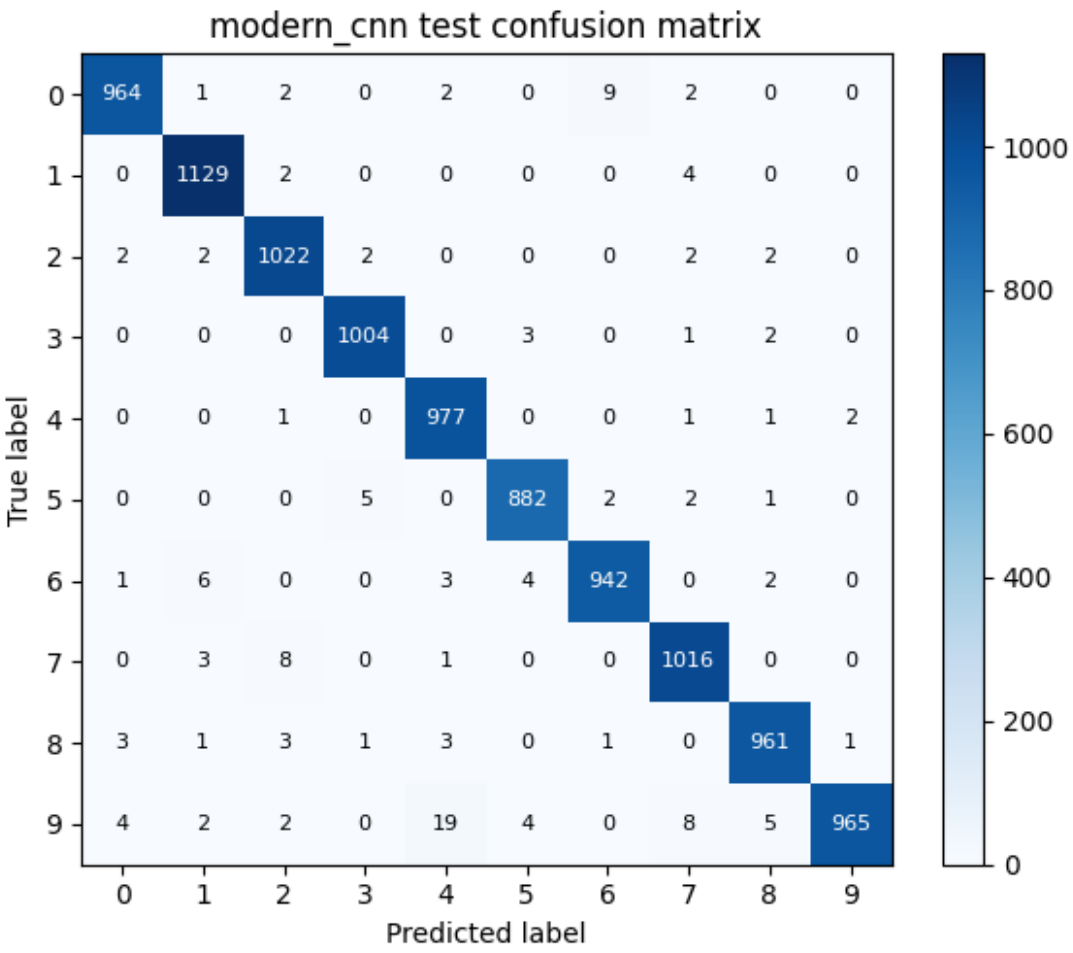





```
modern_cnn parameters: 94186
modern_cnn {'epoch': 1, 'train_loss': 0.2839802600860596,
'train_accuracy': 0.9525666666666667, 'test_loss': 0.2774457073926926,
'test_accuracy': 0.9562}
modern_cnn {'epoch': 2, 'train_loss': 0.11986080509026845,
'train_accuracy': 0.9720666666666666, 'test_loss':
0.11576335766911507, 'test_accuracy': 0.9745}
modern_cnn {'epoch': 3, 'train_loss': 0.09441060029268265,
'train_accuracy': 0.9754, 'test_loss': 0.09130837578773499,
'test_accuracy': 0.975}
modern_cnn {'epoch': 4, 'train_loss': 0.06672879984577497,
'train_accuracy': 0.9818333333333333, 'test_loss':
0.06505732904672623, 'test_accuracy': 0.9823}
modern_cnn {'epoch': 5, 'train_loss': 0.058786685331662494,
'train_accuracy': 0.9838833333333333, 'test_loss':
0.05835170459449291, 'test_accuracy': 0.9846}
modern_cnn {'epoch': 6, 'train_loss': 0.05050719499588013,
'train_accuracy': 0.9853666666666666, 'test_loss': 0.0537502236828208,
'test_accuracy': 0.9847}
modern_cnn {'epoch': 7, 'train_loss': 0.04830480599403381,
```

```
'train_accuracy': 0.9863333333333333, 'test_loss':
0.05085784619823098, 'test_accuracy': 0.9843}
modern_cnn {'epoch': 8, 'train_loss': 0.04028174350857735,
'train_accuracy': 0.9884, 'test_loss': 0.044791576594114306,
'test_accuracy': 0.9867}
modern_cnn {'epoch': 9, 'train_loss': 0.035354738075534506,
'train_accuracy': 0.9900666666666667, 'test_loss':
0.03939474686384201, 'test_accuracy': 0.9881}
modern_cnn {'epoch': 10, 'train_loss': 0.03652269668281078,
'train_accuracy': 0.9894, 'test_loss': 0.04426969992108643,
'test_accuracy': 0.9862}
```






# 3. Transfer learning on ants and bees

I followed the PyTorch transfer-learning tutorial and used ResNet18 with ImageNet1K V1 weights. I replaced its final classifier with a layer that gives two scores: ant and bee.

For fine tuning, all model parameters are updated. For feature extraction, only the new classifier is trained. The pretrained weights and batch-normalization statistics are kept fixed in the second case. Both models start with the same pretrained weights and the same seeded initialization for the new classifier.

I trained each model for 25 epochs using batch size 4 and SGD with learning rate 0.001 and momentum 0.9. The learning rate is multiplied by 0.1 every seven epochs. Training uses random resized crops and horizontal flips. Evaluation uses a fixed resize and center crop. Both use ImageNet normalization.

The dataset provides training and validation folders, so I report validation accuracy rather than calling it test accuracy. The reported results are from epoch 25, not the epoch with the highest validation accuracy. Training accuracy is measured using fixed image crops so it can be compared between the two methods.

```
hym_root=DATA/'hymenoptera_data'
if not hym_root.exists():

download_and_extract_archive('https://download.pytorch.org/tutorial/
hymenoptera_data.zip',
                                    str(DATA),
filename='hymenoptera_data.zip')
normalization=transforms.Normalize([.485,.456,.406],[.229,.224,.225])
aug=transforms.Compose([transforms.RandomResizedCrop(224),transforms.R
andomHorizontalFlip(),
                        transforms.ToTensor(),normalization])
fixed=transforms.Compose([transforms.Resize(256),transforms.CenterCrop
(224),transforms.ToTensor(),normalization])
hym_train=datasets.ImageFolder(hym_root/'train',aug)
hym_train_eval=datasets.ImageFolder(hym_root/'train',fixed)
hym_val=datasets.ImageFolder(hym_root/'val',fixed)
assert hym_train.class_to_idx==hym_val.class_to_idx
print('Classes:',hym_train.classes,'train:',len(hym_train),'validation
:',len(hym_val))

transfer_results={}
for name,frozen in [('fine_tuning',False),
('feature_extraction',True)]:
    seed_all()

model=models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    if frozen:
        for p in model.parameters(): p.requires_grad_(False)
    model.fc=nn.Linear(model.fc.in_features,2)
    model=model.to(device)
    optimizer=torch.optim.SGD([p for p in model.parameters() if
p.requires_grad],lr=.001,momentum=.9)

scheduler=torch.optim.lr_scheduler.StepLR(optimizer,step_size=7,gamma=
.1)
    print(name,'trainable parameters:',sum(p.numel() for p in
model.parameters() if p.requires_grad))

result=train_model(model,loader(hym_train,4,True),loader(hym_train_eva
l,16),loader(hym_val,16),
                       25,optimizer,2,frozen,scheduler,name)
    transfer_results[name]=result
    curves(result[0],name.replace('_', ' ').title(),
eval_label='validation')
    confusion(result[1],hym_train.classes,name+' validation confusion
matrix')
```

```
Downloading https://download.pytorch.org/tutorial/hymenoptera_data.zip
to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\data\
hymenoptera_data.zip
```

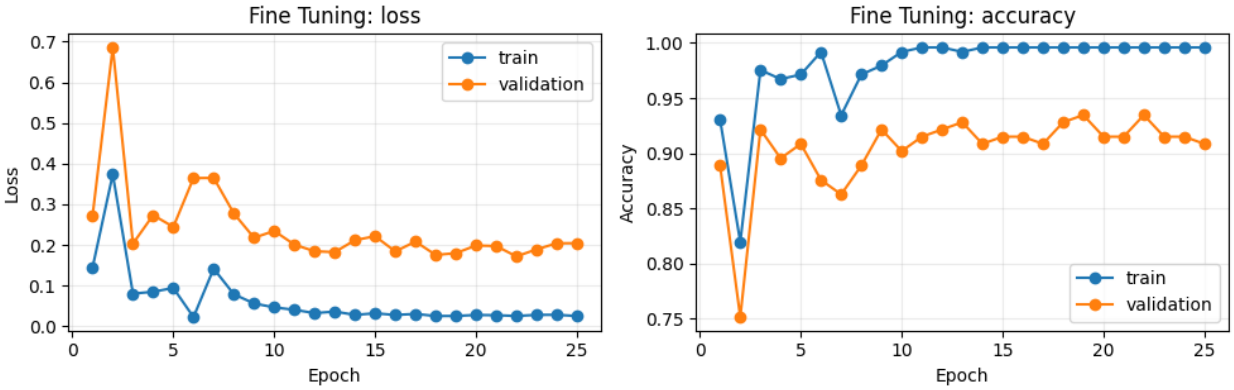
```
100%|██████████| 47.3M/47.3M [00:26<00:00, 1.80MB/s]

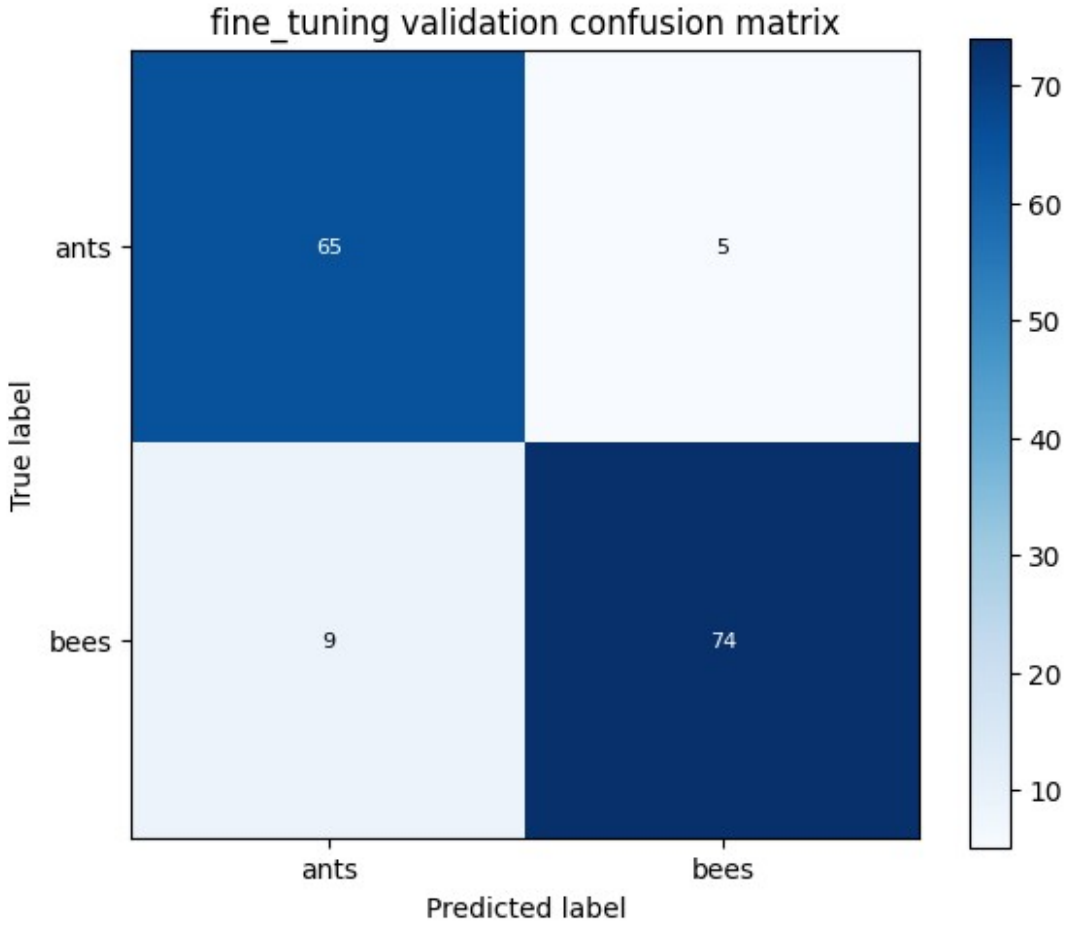
Extracting c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment 3\
data\hymenoptera_data.zip to c:\Users\Chaleesha\Documents\2026\Sem 5\
CV\assignment 3\data
Classes: ['ants', 'bees'] train: 244 validation: 153

Downloading: "https://download.pytorch.org/models/resnet18-
f37072fd.pth" to c:\Users\Chaleesha\Documents\2026\Sem 5\CV\assignment
3\data\torch\hub\checkpoints\resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:24<00:00, 1.93MB/s]

fine_tuning trainable parameters: 11177538
fine_tuning {'epoch': 1, 'train_loss': 0.14521786967506173,
'train_accuracy': 0.930327868852459, 'test_loss': 0.27273500121496863,
'test_accuracy': 0.8888888888888888}
fine_tuning {'epoch': 2, 'train_loss': 0.37372477742896765,
'train_accuracy': 0.819672131147541, 'test_loss': 0.6857649886530209,
'test_accuracy': 0.7516339869281046}
fine_tuning {'epoch': 3, 'train_loss': 0.0803705189285464,
'train_accuracy': 0.9754098360655737, 'test_loss': 0.2034086314291736,
'test_accuracy': 0.9215686274509803}
fine_tuning {'epoch': 4, 'train_loss': 0.08504310531205818,
'train_accuracy': 0.9672131147540983, 'test_loss':
0.27337079500061234, 'test_accuracy': 0.8954248366013072}
fine_tuning {'epoch': 5, 'train_loss': 0.0943237871053766,
'train_accuracy': 0.9713114754098361, 'test_loss':
0.24446535266302769, 'test_accuracy': 0.9084967320261438}
fine_tuning {'epoch': 6, 'train_loss': 0.022766074837475527,
'train_accuracy': 0.9918032786885246, 'test_loss': 0.3644608224139494,
'test_accuracy': 0.8758169934640523}
fine_tuning {'epoch': 7, 'train_loss': 0.14183682297951863,
'train_accuracy': 0.9344262295081968, 'test_loss': 0.364954202782874,
'test_accuracy': 0.8627450980392157}
fine_tuning {'epoch': 8, 'train_loss': 0.07893084470549079,
'train_accuracy': 0.9713114754098361, 'test_loss':
0.27762118897406884, 'test_accuracy': 0.8888888888888888}
fine_tuning {'epoch': 9, 'train_loss': 0.05625780745119345,
'train_accuracy': 0.9795081967213115, 'test_loss':
0.21772153471030442, 'test_accuracy': 0.9215686274509803}
fine_tuning {'epoch': 10, 'train_loss': 0.04662548613230713,
'train_accuracy': 0.9918032786885246, 'test_loss':
0.23460937403385934, 'test_accuracy': 0.9019607843137255}
fine_tuning {'epoch': 11, 'train_loss': 0.040740727340100244,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.20092322117362926, 'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 12, 'train_loss': 0.032495444579446905,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.18455242350989698, 'test_accuracy': 0.9215686274509803}
fine_tuning {'epoch': 13, 'train_loss': 0.03638011214063793,
```

```
'train_accuracy': 0.9918032786885246, 'test_loss':
0.18249491227218528, 'test_accuracy': 0.9281045751633987}
fine_tuning {'epoch': 14, 'train_loss': 0.028203064789537522,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.21188013499079186, 'test_accuracy': 0.9084967320261438}
fine_tuning {'epoch': 15, 'train_loss': 0.032075194290793335,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.22157234109305088, 'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 16, 'train_loss': 0.028030870238044223,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.1842053965026257,
'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 17, 'train_loss': 0.030554161758207885,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.20842149639441296, 'test_accuracy': 0.9084967320261438}
fine_tuning {'epoch': 18, 'train_loss': 0.025386975436914164,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.1755452132692524,
'test_accuracy': 0.9281045751633987}
fine_tuning {'epoch': 19, 'train_loss': 0.025532592271195084,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.17989678281584595, 'test_accuracy': 0.934640522875817}
fine_tuning {'epoch': 20, 'train_loss': 0.027934964013392807,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.19878082493551416, 'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 21, 'train_loss': 0.02736855461643856,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.19744943325815636, 'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 22, 'train_loss': 0.025500052379535846,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.17198798040938534, 'test_accuracy': 0.934640522875817}
fine_tuning {'epoch': 23, 'train_loss': 0.028175298949001264,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.18917841264625002, 'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 24, 'train_loss': 0.028342613354935996,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.2042804095480177,
'test_accuracy': 0.9150326797385621}
fine_tuning {'epoch': 25, 'train_loss': 0.025400068488765935,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.20435893184998455, 'test_accuracy': 0.9084967320261438}
```

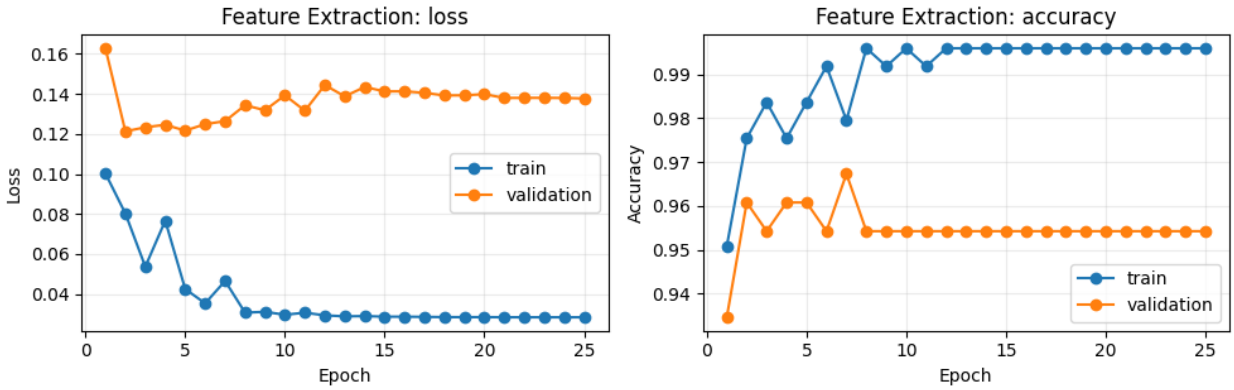


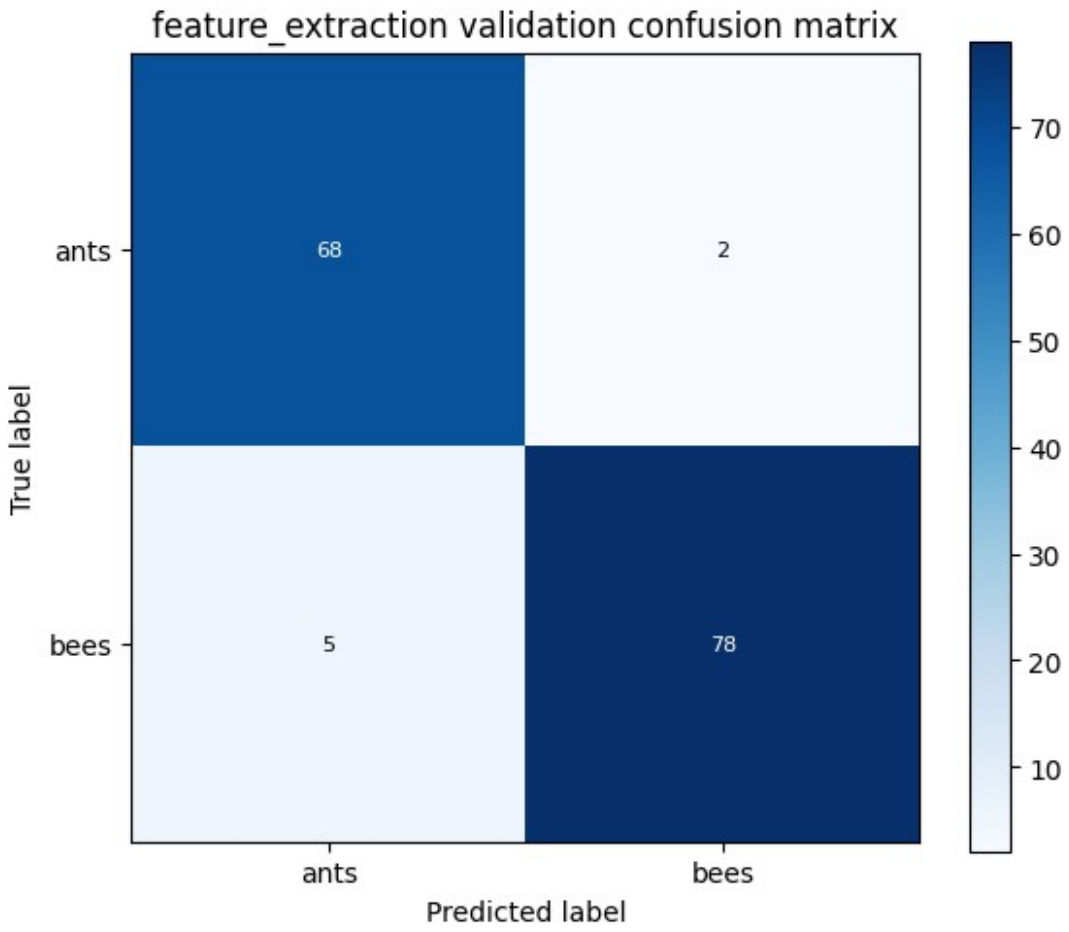




```
feature_extraction trainable parameters: 1026
feature_extraction {'epoch': 1, 'train_loss': 0.10026367680459726,
'train_accuracy': 0.9508196721311475, 'test_loss':
0.16283568097095863, 'test_accuracy': 0.934640522875817}
feature_extraction {'epoch': 2, 'train_loss': 0.08027559547822495,
'train_accuracy': 0.9754098360655737, 'test_loss':
0.12130567723629522, 'test_accuracy': 0.9607843137254902}
feature_extraction {'epoch': 3, 'train_loss': 0.05370035333955874,
```

'train_accuracy': 0.9836065573770492, 'test_loss':
0.12336205033694997, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 4, 'train_loss': 0.07666681091621762,
'train_accuracy': 0.9754098360655737, 'test_loss':
0.12459095192500969, 'test_accuracy': 0.9607843137254902}
feature_extraction {'epoch': 5, 'train_loss': 0.0425323365927964,
'train_accuracy': 0.9836065573770492, 'test_loss':
0.12168560659184176, 'test_accuracy': 0.9607843137254902}
feature_extraction {'epoch': 6, 'train_loss': 0.03537238239631301,
'train_accuracy': 0.9918032786885246, 'test_loss':
0.12491203678978814, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 7, 'train_loss': 0.046548711807757125,
'train_accuracy': 0.9795081967213115, 'test_loss':
0.12643897397066253, 'test_accuracy': 0.9673202614379085}
feature_extraction {'epoch': 8, 'train_loss': 0.030724962562566897,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13435839437970928, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 9, 'train_loss': 0.031126471511164648,
'train_accuracy': 0.9918032786885246, 'test_loss': 0.1318797550949396,
'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 10, 'train_loss': 0.029670307290602903,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.1393407139123655,
'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 11, 'train_loss': 0.030686796993994323,
'train_accuracy': 0.9918032786885246, 'test_loss':
0.13161393542305316, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 12, 'train_loss': 0.029372721391378857,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.14444206528414308, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 13, 'train_loss': 0.028861729489242444,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13882478097685022, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 14, 'train_loss': 0.029003206426735786,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.14347541195894378, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 15, 'train_loss': 0.02870568623918979,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.14127760463290745, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 16, 'train_loss': 0.028703960727472774,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.14132386796614704, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 17, 'train_loss': 0.028599030170284333,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.14055502706883,
'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 18, 'train_loss': 0.028493083158477407,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13926424095833223, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 19, 'train_loss': 0.028460209456379296,
'train_accuracy': 0.9959016393442623, 'test_loss':

```
0.13932234652681288, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 20, 'train_loss': 0.028481743405344057,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13990100614385667, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 21, 'train_loss': 0.028410350506911514,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13799829927145266, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 22, 'train_loss': 0.028406791297382997,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.1380455540675743,
'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 23, 'train_loss': 0.028403473926372217,
'train_accuracy': 0.9959016393442623, 'test_loss': 0.1380590103030984,
'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 24, 'train_loss': 0.02840095895846359,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13799375724169163, 'test_accuracy': 0.954248366013072}
feature_extraction {'epoch': 25, 'train_loss': 0.028400510274728792,
'train_accuracy': 0.9959016393442623, 'test_loss':
0.13776297760165596, 'test_accuracy': 0.954248366013072}
```






## Results and comparison

```
rows=[dict(model='Manual dense CIFAR-
10',split='test',**manual_history[-1])]
for group,split in [(mnist_results,'test'),
(transfer_results,'validation')]:
    for name,(h,cm,seconds) in group.items():
        rows.append(dict(model=name,split=split,seconds=seconds,**h[-
1]))
summary=pd.DataFrame(rows)
summary.to_csv(OUT/'summary.csv',index=False)
display(summary[['model','epoch','train_accuracy','test_accuracy','tra
in_loss','test_loss','split']].style.format(

{'train_accuracy':'{:.2%}','test_accuracy':'{:.2%}','train_loss':'{:.4
f}','test_loss':'{:.4f}'}))

for row in rows:
    gap=100*(row['train_accuracy']-row['test_accuracy'])
    display(Markdown(f"**{row['model']}**: training accuracy
{row['train_accuracy']:.2%}; "
```

```
                    f"{row['split']} accuracy
{row['test_accuracy']:.2%}; "
                    f"generalization gap {gap:.2f} percentage
points."))
lenet=mnist_results['lenet5'][0][-1];
modern=mnist_results['modern_cnn'][0][-1]
display(Markdown(f"The modern CNN changes MNIST test accuracy by "
                 f"{100*(modern['test_accuracy']-
lenet['test_accuracy']):+.2f} percentage points relative to LeNet-5. "
                 "The comparison measures these complete
architectures, so any difference cannot be attributed to batch
normalization or dropout alone."))
for name,(_,cm,_) in mnist_results.items():
    off=cm.copy(); np.fill_diagonal(off,0)
    i,j=np.unravel_index(off.argmax(),off.shape)
    print(f'{name}: most frequent directed error {i} -> {j}:
{off[i,j]} images.')
ft=transfer_results['fine_tuning'][0][-1];
fe=transfer_results['feature_extraction'][0][-1]
display(Markdown(f"Fine tuning changes final validation accuracy by "
                 f"{100*(ft['test_accuracy']-fe['test_accuracy']):
+.2f} percentage points relative to feature extraction. "
                 "Fine tuning adapts the representation but estimates
many more parameters from a small dataset. "
                 "Feature extraction constrains adaptation and reduces
backward computation. "
                 "A single seeded run on this small validation set
does not establish statistical superiority."))
```

```
<pandas.io.formats.style.Styler at 0x11b3a739c90>
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

```
lenet5: most frequent directed error 5 -> 3: 12 images.
modern_cnn: most frequent directed error 9 -> 4: 19 images.
```

```
<IPython.core.display.Markdown object>
```

## Discussion

**Question 1.** The CIFAR-10 test accuracy increased from 41.02% after the first epoch to 48.03% after 10 epochs. Test loss also decreased from 1.707 to 1.512. The final training accuracy was 53.95%, which is 5.92 percentage points above test accuracy. The network learned from the data, but its test accuracy is still fairly low. Unlike a CNN, this network flattens the image and does not directly use the arrangement of neighbouring pixels. This run does not tell us how much each design choice affected the result.

**Question 2.** LeNet-5 reached 98.76% test accuracy, while the modern CNN reached 98.62%. LeNet-5 was slightly better by 0.14 percentage points. Their training accuracies were 99.72% and 98.94%. The modern CNN had a smaller training/test gap, but it also fitted the training data less closely. Its batch normalization and dropout did not lead to higher test accuracy in this run. Since several parts of the architectures differ, the result cannot be explained by dropout or batch normalization alone. LeNet-5's most common mistake was predicting 3 for images of 5 (12 cases). For the modern CNN, it was predicting 4 for images of 9 (19 cases).

**Question 3.** Both transfer-learning models reached 99.59% training accuracy. Fine tuning gave 90.85% validation accuracy, while feature extraction gave 95.42%. Feature extraction was better by 4.58 percentage points. The larger training/validation gap for fine tuning suggests more overfitting under these settings. Keeping the pretrained features fixed worked better on this small dataset. However, there are only 153 validation images, so one prediction changes accuracy by about 0.65 percentage points. These results are from the final epoch and do not prove that feature extraction will always be better.

## Limitations

I used one random seed for each model and did not search for the best hyperparameters. Results may change with another seed, training settings, or hardware. CIFAR-10 and MNIST are different tasks, so their accuracies should not be compared directly.

I recorded test results each epoch as required, but did not use them to tune the models. The ants-and-bees dataset has no separate test set, so the validation results are only an estimate of how the models generalize.